

SLE-3: Architectural Design (Full C4 Model)

Course: 02AML204 – Introduction to Artificial Intelligence

PRN	25UAM107	Name	SUYASH MALI
Division	B	Date	04/10/2026
SLE-2 System	Knight's Shortest Path (BFS vs DFS)	Board	8×8, a1 → h8

1. System Title & Short Description

The Knight's Shortest Path System finds a route for a knight from a1 to h8 on a standard 8×8 chessboard. It compares Breadth-First Search (BFS) and Depth-First Search (DFS) on the same knight-move graph. Each board square is a node and each legal knight move is an edge. The system reconstructs the path and records nodes expanded and execution time so the search behaviour can be compared.

2. Context Diagram (Level 1)



The user supplies or runs the knight-path problem. The system applies BFS or DFS and returns the discovered path together with performance information. For the SLE-2 experiment, the fixed problem was a1 → h8.

3. Container Diagram (Level 2)

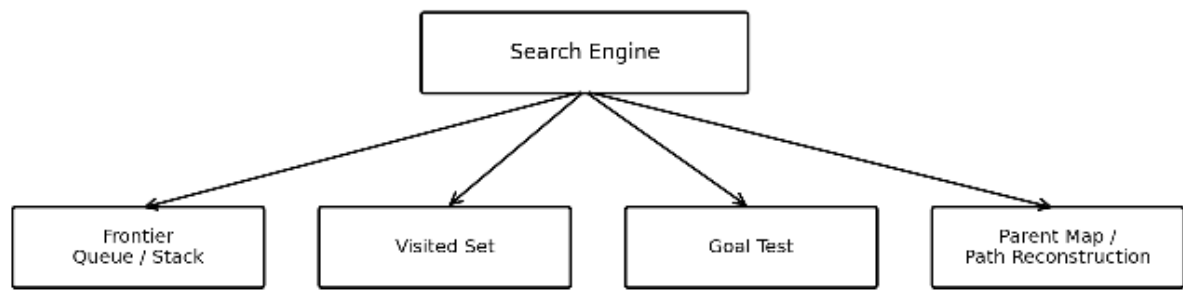


8×8 chessboard • a1 → h8 • legal knight moves

Container	Responsibility
Board Input / Validation	Represents the 8×8 board, start a1 and goal h8, and validates legal positions.
Search Engine (BFS / DFS)	Runs BFS with a queue or DFS with a stack using the same knight-move

	rules.
Visited Set / Frontier	Prevents repeated exploration and stores nodes waiting to be processed.
Path Reconstructor	Uses the parent/came-from map to rebuild the route from h8 back to a1.
Output & Metrics	Reports path length, nodes expanded and measured execution time.

4. Component Diagram (Level 3)



The Search Engine is decomposed into the frontier, visited set, goal test and parent map/path reconstruction. BFS uses a queue and explores nodes level-by-level; DFS uses a stack and follows one branch more deeply before backtracking. The same move-generation and visited logic is used for a fair comparison.

5. Code Level Overview (Level 4)

- knight_moves(square) : returns the legal knight destinations from a square.
- bfs(start, goal) : performs breadth-first search and returns the shortest path.
- dfs(start, goal) : performs depth-first search and returns the first path found.
- reconstruct_path(parent, goal) : rebuilds the final path using the parent map.
- time_algorithm(search_fn) : measures execution time for repeated search runs.
- run_experiment() : runs BFS and DFS and records nodes, path length and timing.

6. Design Decisions

The architecture keeps board representation, search, visited-state handling, path reconstruction and metrics separate so each responsibility is clear. BFS and DFS use the same 8x8 knight graph and start/goal pair, making the comparison consistent. The design stays small enough for a readable four-level C4 model.

7. SLE-2 Results Carried into the Architecture

Metric	BFS	DFS	Observation
Average time (5 runs)	0.0656 ms	0.0573 ms	DFS slightly faster
Best time	0.0604 ms	0.0403 ms	DFS
Worst time	0.0798 ms	0.0821 ms	BFS lower worst-case
Nodes expanded	64	35	DFS fewer in this run
Path length	6 moves	18 moves	BFS is optimal

The SLE-2 experiment used five runs per algorithm. BFS returned the mathematically shortest 6-move route, while DFS found an 18-move route in the measured run. These values are retained here because SLE-3 is the architecture continuation of the same search system.

8. AI Contribution Note

AI tools used: Claude (Anthropic), as recorded in the SLE-2 Contribution Log.

What AI helped with: BFS/DFS knight-move search code, timing/node-counting support, path-visualization diagram generation and report structure.

What I did myself: Chose the knight's shortest-path problem ($a1 \rightarrow h8$), reviewed and ran the experiments, checked the reported nodes/timings/path lengths, and reviewed the architecture and justification.

9. Conclusion

This SLE-3 converts the SLE-2 knight-path search into a complete C4 architecture. The four levels show the system from the user interaction down to containers, search components and important functions. The design also makes the SLE-2 result clear: DFS can be locally faster on this small board, but BFS guarantees the shortest path, which is the key requirement of the knight's shortest-path problem.